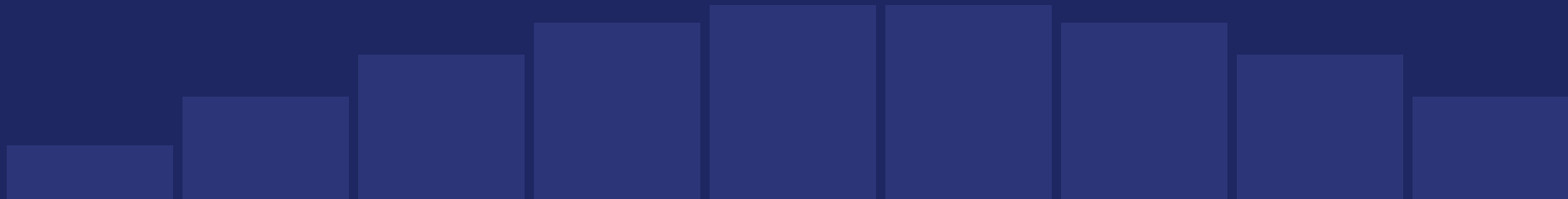


Experiment 5

DPCM & Delta Modulation

Prediction-based redundancy reduction • Granular noise & slope-overload distortion

Objectives: Use prediction to reduce sample redundancy • Identify granular noise and slope-overload distortion



Digital → Analog reconstruction, one step at a time

Objectives



Reduce Sample Redundancy

Use a first-order predictor to exploit the correlation between adjacent samples, so only the unpredictable part of the signal needs to be transmitted.



Identify Distortions

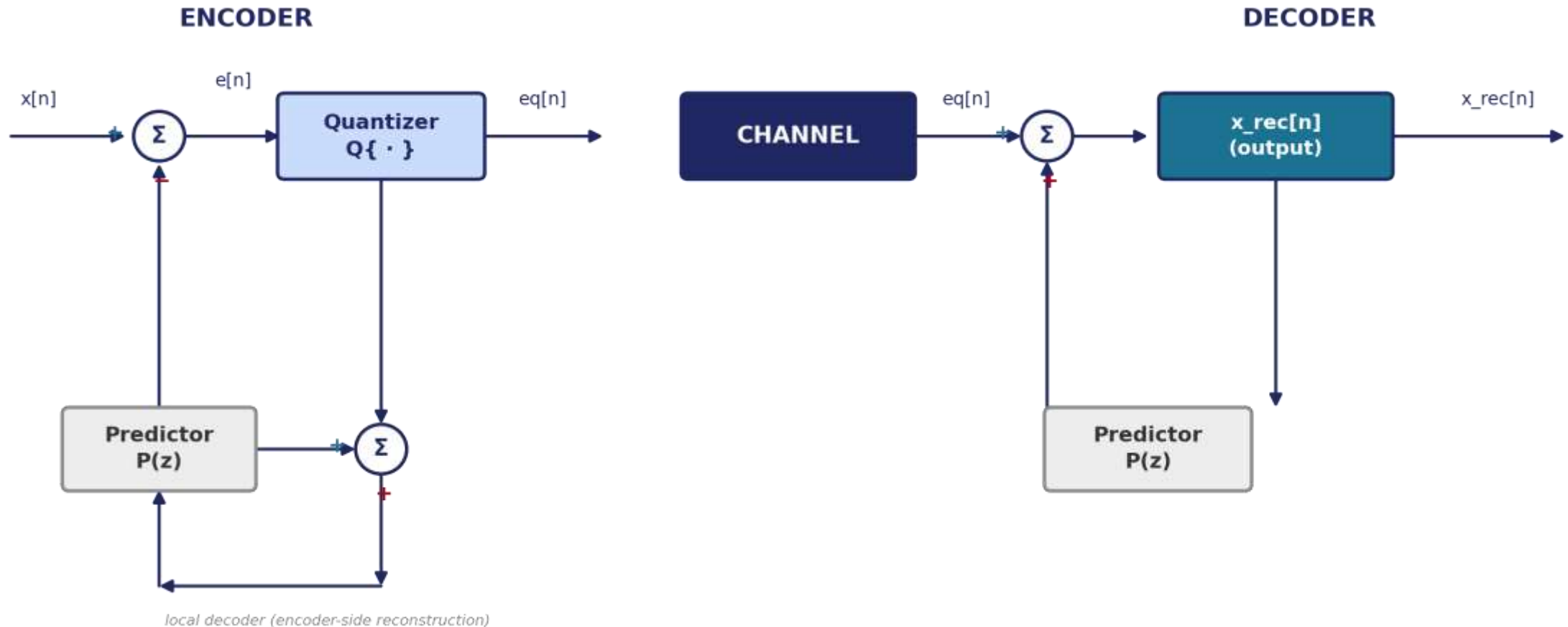
Recognize granular noise (step size too large for a slowly-changing signal) and slope-overload distortion (step size too small for a rapidly-changing signal).

Where this fits in the lab

- **DPCM (Differential Pulse Code Modulation):** encodes the difference between a sample and its predicted value, not the sample itself.
- **Delta Modulation (DM):** the simplest, 1-bit special case of DPCM — the step size Δ is fixed, and only the sign of the error is transmitted.
- **Adaptive Delta Modulation (ADM, optional):** the step size adapts on the fly to balance granular noise against slope overload.

DPCM — System Concept

Instead of quantizing $x[n]$ directly, DPCM quantizes the **prediction error** $e[n] = x[n] - \hat{x}[n]$. Since adjacent samples are correlated, $e[n]$ has much smaller variance than $x[n]$ — so the same number of quantizer levels gives a finer effective step size (lower distortion) than plain PCM.



DPCM — Governing Equations

$$\hat{x}[n] = a_1 \cdot x_{rec}[n - 1]$$

Prediction

First-order predictor: the estimate depends on the previous reconstructed sample, scaled by coefficient $a_1 \leq 1$.

$$e[n] = x[n] - \hat{x}[n]$$

Prediction error

The quantity actually transmitted — the residual left after prediction.

$$e_q[n] = Q\{e[n]\}$$

Quantization

The error is quantized (Task 14) with a uniform quantizer sized to the error's own dynamic range.

$$x_{rec}[n] = \hat{x}[n] + e_q[n]$$

Reconstruction

Decoder (and encoder's local decoder) rebuild the sample by adding back the quantized error.

Delta Modulation — Concept

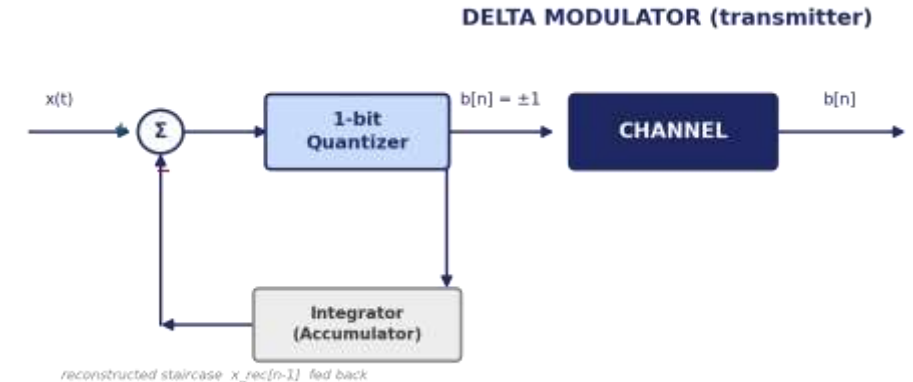
Delta modulation is 1-bit DPCM: the predictor is a simple accumulator, and the quantizer outputs only the **sign** of the error. Each transmitted bit moves the staircase reconstruction up or down by a fixed step Δ .

$$b[n] = \text{sign}(x[n] - x_{rec}[n-1])$$

Comparator + 1-bit quantizer

$$x_{rec}[n] = x_{rec}[n-1] + b[n] \cdot \Delta$$

Integrator (staircase update)



$b[n] = +1 \rightarrow$ step up by Δ

$b[n] = -1 \rightarrow$ step down by Δ

Granular Noise vs. Slope Overload

Granular Noise

- Occurs when Δ is too LARGE for a slowly-varying / flat signal.
- The staircase keeps oscillating by $\pm\Delta$ around the true value even when the input is nearly constant.
- Fix: decrease Δ , or use adaptive step size.

Slope Overload

- Occurs when Δ is too SMALL to track a rapidly-changing signal.
- The staircase cannot keep up with the true signal's slope — a visible lag/ramp error appears.
- Fix: increase Δ , raise sampling rate, or use adaptive step size.

Condition to avoid slope overload:

$$\Delta \cdot f_s \geq \max \left| \frac{dx(t)}{dt} \right|$$

i.e. step $\times$ sample-rate must exceed the input's maximum slope — this is exactly what Task 16/17 tests by sweeping Δ and input frequency.

Implementation Tasks

14

First-order DPCM predictor

Implement $\hat{x}[n] = a_1 \cdot x_{\text{rec}}[n-1]$ with a uniform quantizer on $e[n]$.

15

Compare PCM vs DPCM errors

Quantify prediction-error variance reduction and reconstruction MSE for both schemes.

16

Delta modulation, 3 step sizes

Simulate DM with small, moderate and large Δ and observe the staircase output.

17

Slow & fast test inputs

Repeat DM tests on a low-frequency and a high-frequency signal to expose both distortions.

18

Adaptive delta modulation (optional)

Let Δ grow/shrink automatically based on consecutive bit patterns.

AI-Ready Prompt & Mandatory Validation



AI-READY PROMPT

“Build first-order DPCM and delta-modulation simulations. Explicitly separate prediction, quantization and reconstruction. Demonstrate granular noise and slope overload by varying step size and input frequency.”



MANDATORY VALIDATION

Inspect whether each delta-modulator output step changes by exactly $+\Delta$ or $-\Delta$.

✓ Automated in the code via `validate_delta_steps()` — see Task 16/17 results.

Required Visualizations



Original / Predicted Samples

Overlay of $x[n]$, $\hat{x}[n]$ and $x_{\text{rec}}[n]$ on a common time axis.



Prediction Error

Time-domain plot of $e[n]$, contrasted against the original signal's amplitude.



Delta Staircase

Step-wise reconstruction of the DM output over the original waveform.



MSE vs. Step Size

Sweep of Δ showing the granular-noise / slope-overload trade-off curve.

Python Implementation — DPCM

```
def dpcm_encode_decode(x, levels=16, a1=1.0):
    # Predictor: x_hat[n] = a1 * x_rec[n-1]
    # Error:      e[n]      = x[n] - x_hat[n]
    # Quantizer:  e_q[n]    = Q{ e[n] }
    # Recon:      x_rec[n]  = x_hat[n] + e_q[n]
    N = len(x)
    x_hat = np.zeros(N); e = np.zeros(N)
    e_q = np.zeros(N); x_rec = np.zeros(N)

    for n in range(N):
        x_hat[n] = a1 * x_rec[n-1] if n > 0 else 0.0
        e[n] = x[n] - x_hat[n]
        e_q[n], step = uniform_quantizer(
                                e[n], levels, e_min, e_max)
        x_rec[n] = x_hat[n] + e_q[n]
    return x_hat, e, e_q, x_rec
```

Key points

- Predictor coefficient $a_1 = 0.95$ (< 1 for loop stability).
- Quantizer range is matched to the prediction error's own span — not the full signal range.
- Encoder and decoder use the **SAME** reconstructed samples → no drift (closed-loop / “local decoder” design).
- Full source: `dpcm_delta_modulation.py` on GitHub.

Python Implementation — Delta Modulation

```
def delta_modulation(x, delta):
    N = len(x)
    x_rec = np.zeros(N); bits = np.zeros(N, int)
    for n in range(1, N):
        if x[n] >= x_rec[n-1]:
            bits[n] = 1; x_rec[n] = x_rec[n-1] + delta
        else:
            bits[n] = -1; x_rec[n] = x_rec[n-1] - delta
    return x_rec, bits

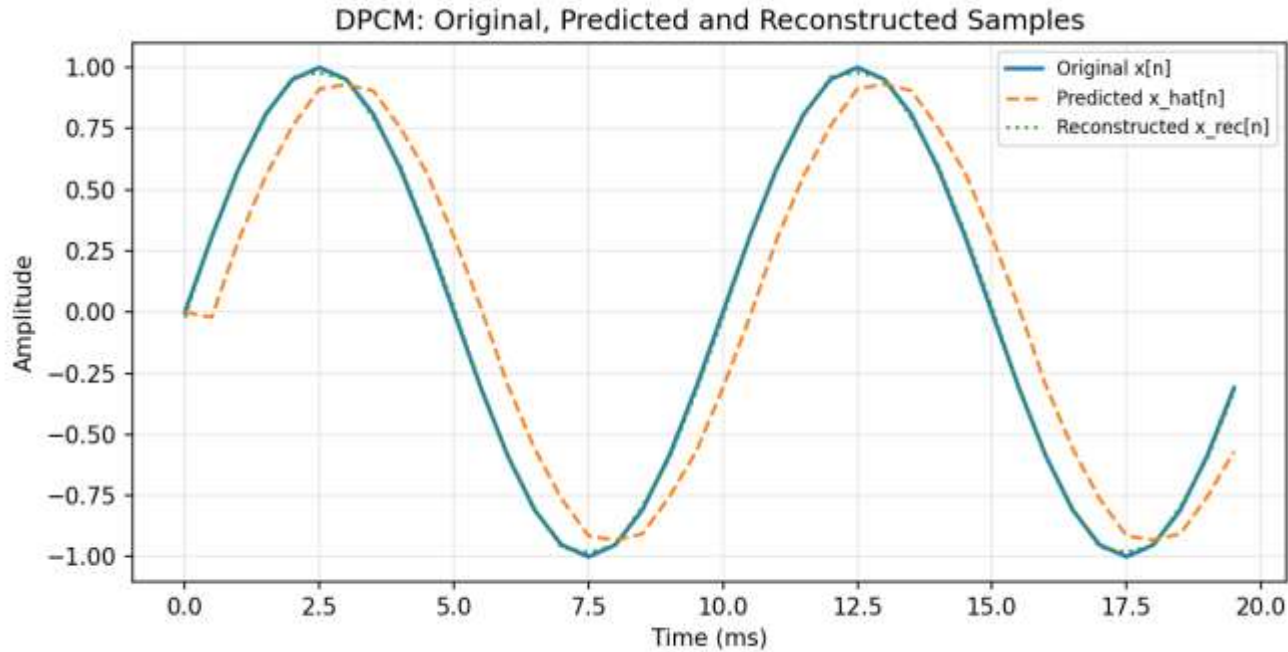
def validate_delta_steps(x_rec, delta):
    diffs = np.diff(x_rec)
    ok = np.all(np.isclose(np.abs(diffs), delta))
    print('PASS' if ok else 'FAIL')    # mandatory check
```

Optional: Adaptive DM

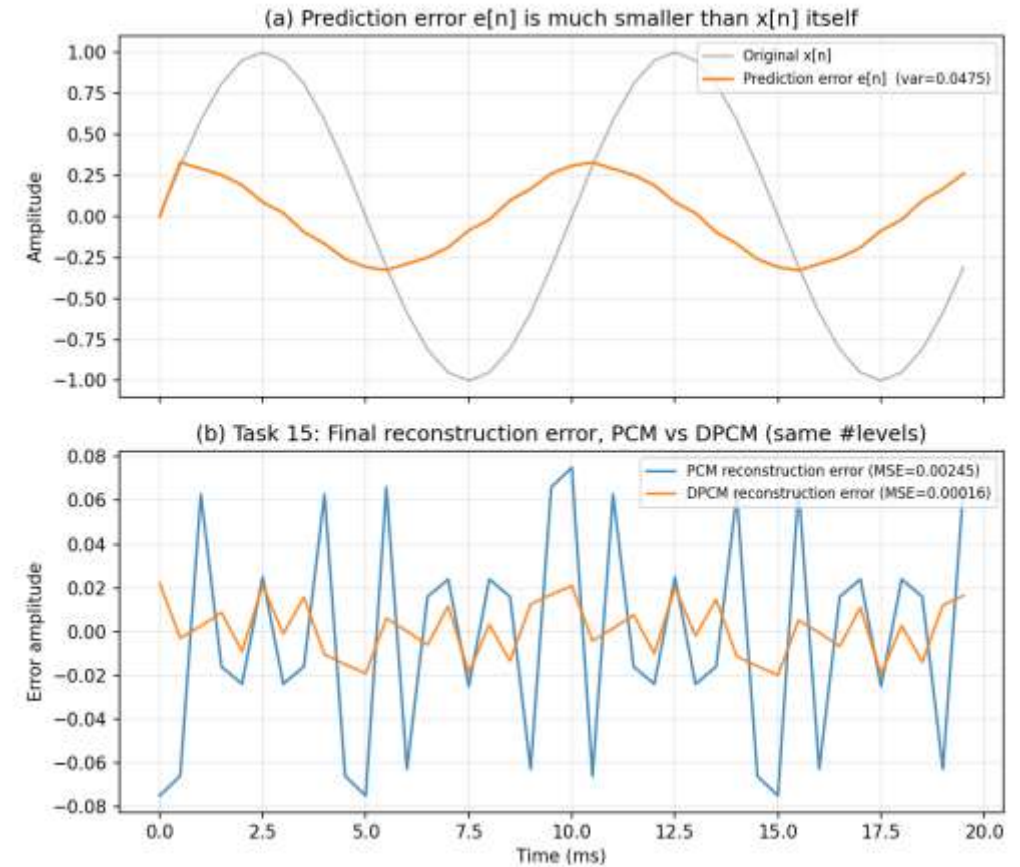
- Step grows ($\times k$) on repeated same-sign bits $\rightarrow$ recovers quickly from slope overload.
- Step shrinks on bit reversal $\rightarrow$ suppresses granular noise on flat regions.
- Bounded to $[\Delta_{\min}, \Delta_{\max}]$ for stability.
- Implemented as `adaptive_delta_modulation()` in the script.

✓ Every run PASSES: $|x_{\text{rec}}[n] - x_{\text{rec}}[n-1]| = \Delta$ exactly.

Results — DPCM Prediction & Error

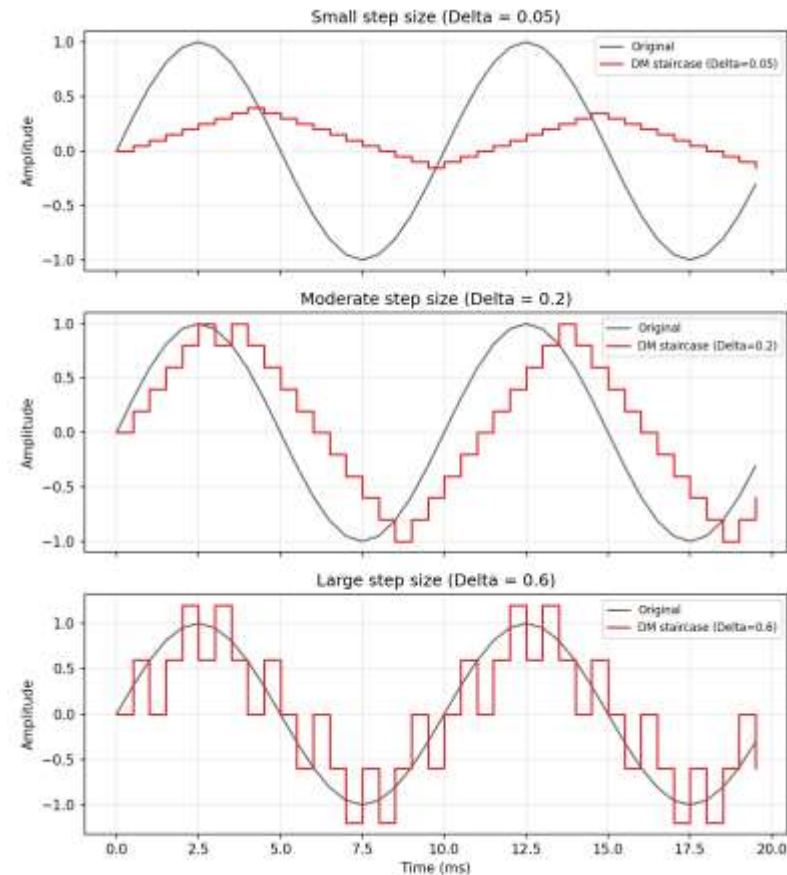


$\text{var}(x) = 0.500 \rightarrow \text{var}(e) = 0.048$ (90% redundancy removed by the predictor)

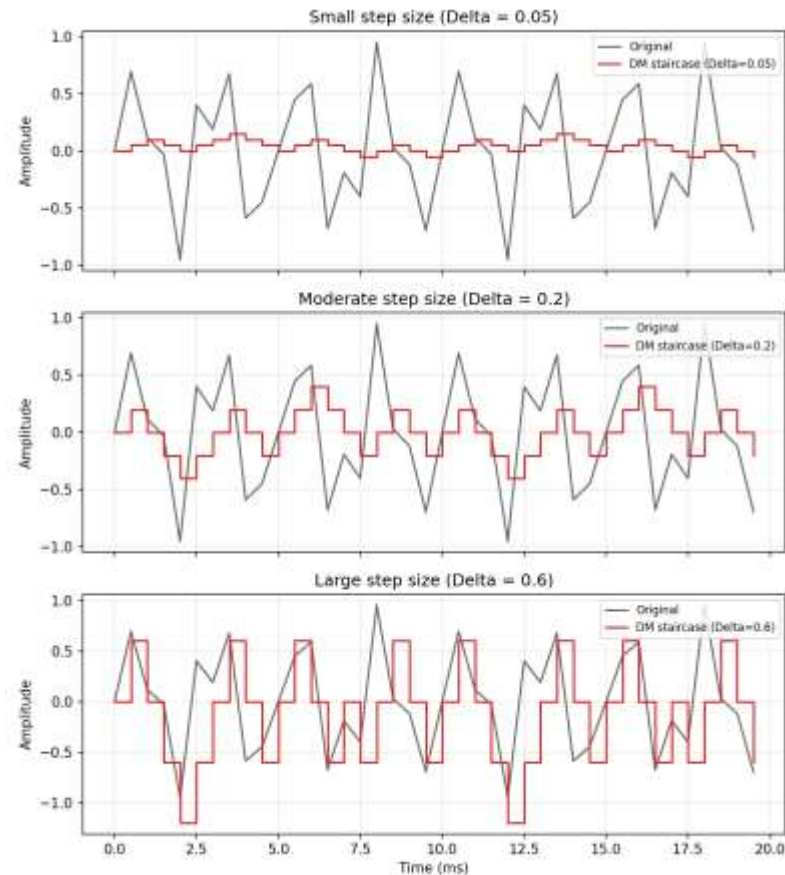


Results — Delta Modulation

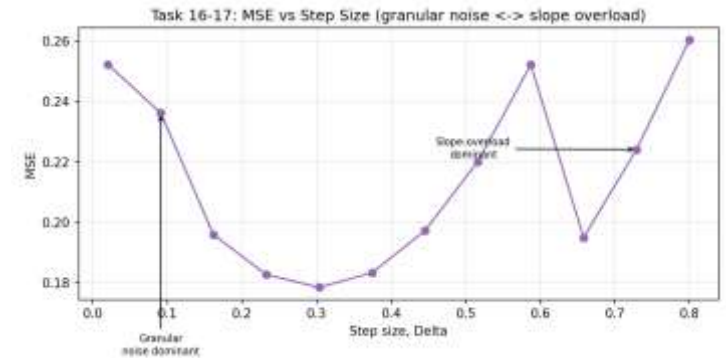
Slow (100 Hz) input



Fast (400+900 Hz) input



MSE vs. Δ (fast input)



- Small Δ : slope overload dominates on the slow tone.
- Large Δ : granular noise dominates near flat/turning points.
- Optimum $\Delta \approx 0.30$ minimizes MSE for the fast test signal.

Observation & Interpretation

Before each run: write down the expected physical effect of the parameter change.
After each run: state whether the observation agrees with theory; if not, identify the discrepancy and the diagnostic test used.

Validation results from this run

Test	Δ	Step check ($\pm\Delta$ exactly)
Slow input — small step	0.05	PASS
Slow input — moderate step	0.20	PASS
Slow input — large step	0.60	PASS
Fast input — small step	0.05	PASS
Fast input — moderate step	0.20	PASS
Fast input — large step	0.60	PASS

Discrepancy checklist

- Staircase lags the peak → slope overload.
- Staircase jitters on a flat segment → granular noise.
- Step $\neq \Delta$ in the log → bug in the accumulator, not the theory.

SUMMARY

Conclusion

- DPCM's first-order predictor cut prediction-error variance to $\sim 10\%$ of the original signal variance, lowering reconstruction MSE for the same bit budget as PCM.
- Delta modulation clearly showed slope overload at small Δ and granular noise at large Δ , with a measurable optimum step size.
- Every DM output step validated to exactly $\pm\Delta$, confirming a correct implementation.
- Adaptive DM reduced sensitivity to a single fixed Δ across both slow and fast segments.

Code & Assets

- **dpcm_delta_modulation.py**
- requirements.txt
- README.md (setup + task map)
- figs/ — all required plots

Ready to push to GitHub